jongwook using sdpa if available (#2359) 27f9713 · 6 months ago

345 lines (282 loc) · 11.5 KB

Code Blame Raw

```python
import gzip
from contextlib import contextmanager
from dataclasses import dataclass
from typing import Dict, Iterable, Optional, Tuple

import numpy as np
import torch
import torch.nn.functional as F
from torch import Tensor, nn

from .decoding import decode as decode_function
from .decoding import detect_language as detect_language_function
from .transcribe import transcribe as transcribe_function

try:
    from torch.nn.functional import scaled_dot_product_attention

    SDPA_AVAILABLE = True
except (ImportError, RuntimeError, OSError):
    scaled_dot_product_attention = None
    SDPA_AVAILABLE = False


@dataclass
class ModelDimensions:
    n_mels: int
    n_audio_ctx: int
    n_audio_state: int
```

```
        n_audio_head: int
        n_audio_layer: int
        n_vocab: int
        n_text_ctx: int
        n_text_state: int
        n_text_head: int
        n_text_layer: int


    class LayerNorm(nn.LayerNorm):
        def forward(self, x: Tensor) -> Tensor:
            return super().forward(x.float()).type(x.dtype)


    class Linear(nn.Linear):
        def forward(self, x: Tensor) -> Tensor:
            return F.linear(
                x,
                self.weight.to(x.dtype),
                None if self.bias is None else self.bias.to(x.dtype),
            )


    class Conv1d(nn.Conv1d):
        def _conv_forward(
            self, x: Tensor, weight: Tensor, bias: Optional[Tensor]
        ) -> Tensor:
            return super()._conv_forward(
                x, weight.to(x.dtype), None if bias is None else bias.to(x.dtype)
            )


    def sinusoids(length, channels, max_timescale=10000):
        """Returns sinusoids for positional embedding"""
        assert channels % 2 == 0
        log_timescale_increment = np.log(max_timescale) / (channels // 2 - 1)
        inv_timescales = torch.exp(-log_timescale_increment * torch.arange(channels // 2))
        scaled_time = torch.arange(length)[:, np.newaxis] * inv_timescales[np.newaxis, :]
        return torch.cat([torch.sin(scaled_time), torch.cos(scaled_time)], dim=1)


    @contextmanager
    def disable_sdpa():
        prev_state = MultiHeadAttention.use_sdpa
        try:
            MultiHeadAttention.use_sdpa = False
            yield
        finally:
            MultiHeadAttention.use_sdpa = prev_state


    class MultiHeadAttention(nn.Module):
        use_sdpa = True
```

```
    def __init__(self, n_state: int, n_head: int):
        super().__init__()
        self.n_head = n_head
        self.query = Linear(n_state, n_state)
        self.key = Linear(n_state, n_state, bias=False)
        self.value = Linear(n_state, n_state)
        self.out = Linear(n_state, n_state)

    def forward(
        self,
        x: Tensor,
        xa: Optional[Tensor] = None,
        mask: Optional[Tensor] = None,
        kv_cache: Optional[dict] = None,
    ):
        q = self.query(x)

        if kv_cache is None or xa is None or self.key not in kv_cache:
            # hooks, if installed (i.e. kv_cache is not None), will prepend the cached kv tenso
            # otherwise, perform key/value projections for self- or cross-attention as usual.
            k = self.key(x if xa is None else xa)
            v = self.value(x if xa is None else xa)
        else:
            # for cross-attention, calculate keys and values once and reuse in subsequent calls
            k = kv_cache[self.key]
            v = kv_cache[self.value]

        wv, qk = self.qkv_attention(q, k, v, mask)
        return self.out(wv), qk

    def qkv_attention(
        self, q: Tensor, k: Tensor, v: Tensor, mask: Optional[Tensor] = None
    ) -> Tuple[torch.Tensor, Optional[torch.Tensor]]:
        n_batch, n_ctx, n_state = q.shape
        scale = (n_state // self.n_head) ** -0.25
        q = q.view(*q.shape[:2], self.n_head, -1).permute(0, 2, 1, 3)
        k = k.view(*k.shape[:2], self.n_head, -1).permute(0, 2, 1, 3)
        v = v.view(*v.shape[:2], self.n_head, -1).permute(0, 2, 1, 3)

        if SDPA_AVAILABLE and MultiHeadAttention.use_sdpa:
            a = scaled_dot_product_attention(
                q, k, v, is_causal=mask is not None and n_ctx > 1
            )
            out = a.permute(0, 2, 1, 3).flatten(start_dim=2)
            qk = None
        else:
            qk = (q * scale) @ (k * scale).transpose(-1, -2)
            if mask is not None:
                qk = qk + mask[:n_ctx, :n_ctx]
            qk = qk.float()

            w = F.softmax(qk, dim=-1).to(q.dtype)
            out = (w @ v).permute(0, 2, 1, 3).flatten(start_dim=2)
            qk = qk.detach()

```

```
            return out, qk


    class ResidualAttentionBlock(nn.Module):
        def __init__(self, n_state: int, n_head: int, cross_attention: bool = False):
            super().__init__()

            self.attn = MultiHeadAttention(n_state, n_head)
            self.attn_ln = LayerNorm(n_state)

            self.cross_attn = (
                MultiHeadAttention(n_state, n_head) if cross_attention else None
            )
            self.cross_attn_ln = LayerNorm(n_state) if cross_attention else None

            n_mlp = n_state * 4
            self.mlp = nn.Sequential(
                Linear(n_state, n_mlp), nn.GELU(), Linear(n_mlp, n_state)
            )
            self.mlp_ln = LayerNorm(n_state)

        def forward(
            self,
            x: Tensor,
            xa: Optional[Tensor] = None,
            mask: Optional[Tensor] = None,
            kv_cache: Optional[dict] = None,
        ):
            x = x + self.attn(self.attn_ln(x), mask=mask, kv_cache=kv_cache)[0]
            if self.cross_attn:
                x = x + self.cross_attn(self.cross_attn_ln(x), xa, kv_cache=kv_cache)[0]
            x = x + self.mlp(self.mlp_ln(x))
            return x


    class AudioEncoder(nn.Module):
        def __init__(
            self, n_mels: int, n_ctx: int, n_state: int, n_head: int, n_layer: int
        ):
            super().__init__()
            self.conv1 = Conv1d(n_mels, n_state, kernel_size=3, padding=1)
            self.conv2 = Conv1d(n_state, n_state, kernel_size=3, stride=2, padding=1)
            self.register_buffer("positional_embedding", sinusoids(n_ctx, n_state))

            self.blocks: Iterable[ResidualAttentionBlock] = nn.ModuleList(
                [ResidualAttentionBlock(n_state, n_head) for _ in range(n_layer)]
            )
            self.ln_post = LayerNorm(n_state)

        def forward(self, x: Tensor):
            """
            x : torch.Tensor, shape = (batch_size, n_mels, n_ctx)
                the mel spectrogram of the audio
            """
```

```
            x = F.gelu(self.conv1(x))
            x = F.gelu(self.conv2(x))
            x = x.permute(0, 2, 1)

            assert x.shape[1:] == self.positional_embedding.shape, "incorrect audio shape"
            x = (x + self.positional_embedding).to(x.dtype)

            for block in self.blocks:
                x = block(x)

            x = self.ln_post(x)
            return x


    class TextDecoder(nn.Module):
        def __init__(
            self, n_vocab: int, n_ctx: int, n_state: int, n_head: int, n_layer: int
        ):
            super().__init__()

            self.token_embedding = nn.Embedding(n_vocab, n_state)
            self.positional_embedding = nn.Parameter(torch.empty(n_ctx, n_state))

            self.blocks: Iterable[ResidualAttentionBlock] = nn.ModuleList(
                [
                    ResidualAttentionBlock(n_state, n_head, cross_attention=True)
                    for _ in range(n_layer)
                ]
            )
            self.ln = LayerNorm(n_state)

            mask = torch.empty(n_ctx, n_ctx).fill_(-np.inf).triu_(1)
            self.register_buffer("mask", mask, persistent=False)

        def forward(self, x: Tensor, xa: Tensor, kv_cache: Optional[dict] = None):
            """
            x : torch.LongTensor, shape = (batch_size, <= n_ctx)
                the text tokens
            xa : torch.Tensor, shape = (batch_size, n_audio_ctx, n_audio_state)
                the encoded audio features to be attended on
            """
            offset = next(iter(kv_cache.values())).shape[1] if kv_cache else 0
            x = (
                self.token_embedding(x)
                + self.positional_embedding[offset : offset + x.shape[-1]]
            )
            x = x.to(xa.dtype)

            for block in self.blocks:
                x = block(x, xa, mask=self.mask, kv_cache=kv_cache)

            x = self.ln(x)
            logits = (
                x @ torch.transpose(self.token_embedding.weight.to(x.dtype), 0, 1)
```

```
247             ).float()
248
249             return logits
250
251
252     class Whisper(nn.Module):
253         def __init__(self, dims: ModelDimensions):
254             super().__init__()
255             self.dims = dims
256             self.encoder = AudioEncoder(
257                 self.dims.n_mels,
258                 self.dims.n_audio_ctx,
259                 self.dims.n_audio_state,
260                 self.dims.n_audio_head,
261                 self.dims.n_audio_layer,
262             )
263             self.decoder = TextDecoder(
264                 self.dims.n_vocab,
265                 self.dims.n_text_ctx,
266                 self.dims.n_text_state,
267                 self.dims.n_text_head,
268                 self.dims.n_text_layer,
269             )
270             # use the last half among the decoder layers for time alignment by default;
271             # to use a specific set of heads, see `set_alignment_heads()` below.
272             all_heads = torch.zeros(
273                 self.dims.n_text_layer, self.dims.n_text_head, dtype=torch.bool
274             )
275             all_heads[self.dims.n_text_layer // 2 :] = True
276             self.register_buffer("alignment_heads", all_heads.to_sparse(), persistent=False)
277
278         def set_alignment_heads(self, dump: bytes):
279             array = np.frombuffer(
280                 gzip.decompress(base64.b85decode(dump)), dtype=bool
281             ).copy()
282             mask = torch.from_numpy(array).reshape(
283                 self.dims.n_text_layer, self.dims.n_text_head
284             )
285             self.register_buffer("alignment_heads", mask.to_sparse(), persistent=False)
286
287         def embed_audio(self, mel: torch.Tensor):
288             return self.encoder(mel)
289
290         def logits(self, tokens: torch.Tensor, audio_features: torch.Tensor):
291             return self.decoder(tokens, audio_features)
292
293         def forward(
294             self, mel: torch.Tensor, tokens: torch.Tensor
295         ) -> Dict[str, torch.Tensor]:
296             return self.decoder(tokens, self.encoder(mel))
297
298         @property
299         def device(self):
300             return next(self.parameters()).device
```

```
301
302         @property
303         def is_multilingual(self):
304             return self.dims.n_vocab >= 51865
305
306         @property
307         def num_languages(self):
308             return self.dims.n_vocab - 51765 - int(self.is_multilingual)
309
310         def install_kv_cache_hooks(self, cache: Optional[dict] = None):
311             """
312             The `MultiHeadAttention` module optionally accepts `kv_cache` which stores the key and
313             tensors calculated for the previous positions. This method returns a dictionary that st
314             all caches, and the necessary hooks for the key and value projection modules that save
315             intermediate tensors to be reused during later calculations.
316
317             Returns
318             -------
319             cache : Dict[nn.Module, torch.Tensor]
320                 A dictionary object mapping the key/value projection modules to its cache
321             hooks : List[RemovableHandle]
322                 List of PyTorch RemovableHandle objects to stop the hooks to be called
323             """
324             cache = {**cache} if cache is not None else {}
325             hooks = []
326
327             def save_to_cache(module, _, output):
328                 if module not in cache or output.shape[1] > self.dims.n_text_ctx:
329                     # save as-is, for the first token or cross attention
330                     cache[module] = output
331                 else:
332                     cache[module] = torch.cat([cache[module], output], dim=1).detach()
333                 return cache[module]
334
335             def install_hooks(layer: nn.Module):
336                 if isinstance(layer, MultiHeadAttention):
337                     hooks.append(layer.key.register_forward_hook(save_to_cache))
338                     hooks.append(layer.value.register_forward_hook(save_to_cache))
339
340             self.decoder.apply(install_hooks)
341             return cache, hooks
342
343         detect_language = detect_language_function
344         transcribe = transcribe_function
345         decode = decode_function
```